Practical 09

1.

AIM:

Counter race: several threads each increment a shared counter many times. Run it

without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.

CODE:

```
class Counter {
int count = 0;

   void increment() {
count++;
   }

   synchronized void synchronizedIncrement() {
count++;
   }
}

public class CounterRace {     static final
int THREADS = 10;     static final int
INCREMENTS = 100000;

   public static void main(String[] args) throws InterruptedException {
Counter c1 = new Counter();
```

```
        Thread[] t1 = new Thread[THREADS];

        for (int i = 0; i < THREADS; i++) {
t1[i] = new Thread(() -> {
                for (int j = 0; j < INCREMENTS; j++) {
c1.increment();
                }
});
t1[i].start();
        }

        for (int i = 0; i < THREADS; i++) {
            t1[i].join();
        }

        System.out.println("Without Synchronization: " + c1.count);
        System.out.println("Expected Count: " + (THREADS *
INCREMENTS));

        Counter c2 = new Counter();
        Thread[] t2 = new Thread[THREADS];

        for (int i = 0; i < THREADS; i++) {
t2[i] = new Thread(() -> {
                for (int j = 0; j < INCREMENTS; j++) {
c2.synchronizedIncrement();
                }
```

```
            });
t2[i].start();
        }

        for (int i = 0; i < THREADS; i++) {
            t2[i].join();
        }

        System.out.println("With Synchronization: " + c2.count);
        System.out.println("Expected Count: " + (THREADS *
INCREMENTS));
    }
}
```

OUTPUT :

```
erRace.java } ; if ($?) { java CounterRace }
Without Synchronization: 293685
Expected Count: 1000000
With Synchronization: 1000000
Expected Count: 1000000
```

2.

AIM:

Seat booking race: a shared seatsLeft of 5 with about 10 threads each trying to book one

seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized

book() exactly 5 do.

CODE:

```
class SeatBooking {
int seatsLeft = 5;     int
```

```
successfulBookings =
0;


    void bookWithoutSync() {
        if (seatsLeft > 0) {
try {
                Thread.sleep(10);
            } catch (InterruptedException e) {
Thread.currentThread().interrupt();
                return;
            }

            seatsLeft--;
successfulBookings++;
            System.out.println(Thread.currentThread().getName() + " booked
a seat");
        } else {
            System.out.println(Thread.currentThread().getName() + " failed");
        }
    }

    synchronized void book() {
if (seatsLeft > 0) {
seatsLeft--;
successfulBookings++;
            System.out.println(Thread.currentThread().getName() + " booked
a seat");
        } else {
```

```
            System.out.println(Thread.currentThread().getName() + " failed");
}
    }
}

public class SeatBookingRace {
    public static void main(String[] args) throws InterruptedException {
        SeatBooking b1 = new SeatBooking();
        Thread[] t1 = new Thread[10];

        System.out.println("Without Synchronization:");

        for (int i = 0; i < 10; i++) {
            t1[i] = new Thread(() -> b1.bookWithoutSync(), "Thread-" + (i +
1));
            t1[i].start();
        }

        for (int i = 0; i < 10; i++) {
t1[i].join();
        }

        System.out.println("Successful bookings: " +
b1.successfulBookings);
        System.out.println("Seats left: " + b1.seatsLeft);

        SeatBooking b2 = new SeatBooking();
        Thread[] t2 = new Thread[10];
```

```
        System.out.println("\nWith Synchronization:");

        for (int i = 0; i < 10; i++) {
            t2[i] = new Thread(() -> b2.book(), "Thread-" + (i + 1));
            t2[i].start();
        }

        for (int i = 0; i < 10; i++) {
t2[i].join();
        }

        System.out.println("Successful bookings: " +
b2.successfulBookings);
        System.out.println("Seats left: " + b2.seatsLeft);
    }
}
```

OUTPUT :

```
ookingRace.java } ; if ($?) { java SeatBookingRace }
Successful bookings: 6
Seats left: 0

With Synchronization:
Thread-1 booked a seat
Thread-2 booked a seat
Thread-6 booked a seat
Thread-3 booked a seat
Thread-10 booked a seat
Thread-5 failed
Thread-7 failed
Thread-8 failed
Thread-4 failed
Thread-9 failed
Successful bookings: 5
Seats left: 0
```

3.

AIM:

(For Advanced Learners – home) Split a large array of numbers across several threads

that add into one shared total, show the wrong result, then fix it two ways and compare; decide how to measure the difference.

CODE:

```
import java.util.concurrent.atomic.AtomicLong;


class SharedTotal {
long total = 0;


    void add(long value) {
total += value;
    }


    synchronized void addSync(long value) {
total += value;
    }
}


public class ParallelSum {
static final int SIZE = 1000000;
static final int THREADS = 10;
static long[] arr = new
long[SIZE];
```

```
   static void runTest(int mode) throws InterruptedException {
       SharedTotal shared = new SharedTotal();
       AtomicLong atomicTotal = new AtomicLong(0);

Thread[] threads = new Thread[THREADS];

       long start = System.nanoTime();

       for (int i = 0; i < THREADS; i++) {            final
int startIndex = i * (SIZE / THREADS);
final int endIndex = (i == THREADS - 1)
? SIZE : (i + 1) * (SIZE / THREADS);

           threads[i] = new Thread(() -> {
long localSum = 0;

               for (int j = startIndex; j < endIndex; j++) {
localSum += arr[j];
               }

               if (mode == 0) {
shared.add(localSum);               } else
if (mode == 1) {
shared.addSync(localSum);
               } else {
                   atomicTotal.addAndGet(localSum);
```

```
                }
});

            threads[i].start();
        }

        for (Thread t : threads) {
            t.join();
}

        long end = System.nanoTime();

        if (mode == 0) {
            System.out.println("Without synchronization: " + shared.total);
        } else if (mode == 1) {
            System.out.println("With synchronized: " + shared.total);
        } else {
            System.out.println("With AtomicLong: " + atomicTotal.get());
        }

        System.out.println("Time: " + (end - start) / 1_000_000.0 + " ms");
    }

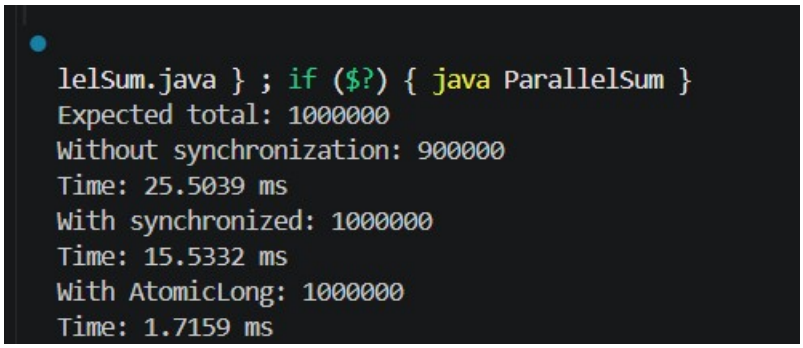
    public static void main(String[] args) throws InterruptedException {
for (int i = 0; i < SIZE; i++) {            arr[i] = 1;
```

```
        }

        System.out.println("Expected total: " + SIZE);

        runTest(0);
runTest(1);
runTest(2);
    }
}
```

OUTPUT:



pg.